

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 4.2

МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ И РЕГРЕССИЯ С ИСПОЛЬЗОВАНИЕМ НЕЙРОННЫХ СЕТЕЙ KERAS

(наборы данных Reuters и Boston Housing)

Продолжительность: 2 академических часа

Необходимые инструменты: Python, Jupyter Notebook, TensorFlow/Keras, Matplotlib

Уровень: базовый – продолжение ПЗ 4.1

1. Цели работы

После выполнения практического занятия студент должен уметь:

По части А (многоклассовая классификация):

- загружать набор данных Reuters из Keras;
- выполнять векторизацию текстовых данных;
- выполнять one-hot кодирование меток классов;
- строить и обучать модель с выходным слоем softmax;
- интерпретировать результаты модели для многоклассовой классификации;
- строить графики изменения потерь и точности и анализировать переобучение.

По части В (регрессия):

- загружать набор данных Boston Housing;
- делать нормализацию признаков;
- строить модель для регрессии (линейный выход, функции потерь mse/mae);
- выполнять k-fold кросс-валидацию для подбора числа эпох;
- оценивать качество модели по метрике MAE;
- делать прогнозы и интерпретировать результаты модели.

2. Теоретическое введение

2.1. Многоклассовая классификация (softmax)

Многоклассовая классификация — это задача, в которой каждый входной объект должен быть отнесён к **одному из нескольких** классов. В наборе данных Reuters таких классов **46**, и каждый текст относится к одному классу (теме).

В отличие от бинарной классификации:

- выходной слой имеет размер K (число классов),
- функция активации — `softmax`,
- функция потерь — `categorical_crossentropy`,
- метки обязательно должны быть либо `one-hot`, либо целочисленными с использованием `sparse_categorical_crossentropy`.

2.2. Регрессия

Регрессия — это задача прогноза **непрерывного числового значения**. В Boston Housing целевая переменная — **средняя стоимость жилья в районе (в тысячах долларов)**.

Особенности:

- выходной слой один, без активации (линейный выход);
- функции потерь: `mse`, `mae`;
- нормализация входных признаков обязательна.

2.3. k-fold кросс-валидация

Проблема Boston Housing — очень маленький набор данных. Чтобы корректно оценить качество модели, используется `k-fold` метод:

- обучаем модель k раз на разных подвыборках,
- усредняем результаты,
- выбираем число эпох, минимизирующее ошибку.

3. Практическая часть А — Многоклассовая классификация (Reuters)

Шаг 1. Загрузка набора данных Reuters

Keras предоставляет полностью подготовленный набор данных:

```
from tensorflow.keras.datasets import reuters
```

```
(train_data, train_labels), (test_data, test_labels) =  
reuters.load_data(num_words=10000)
```

Что получает студент:

- `train_data` — список статей, каждая в виде последовательности индексов слов;
- `train_labels` — список категорий (от 0 до 45).

Шаг 2. Векторизация текста

Необходимо преобразовать последовательности в векторы длины 10000:

```
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, seq in enumerate(sequences):
        results[i, seq] = 1
    return results

x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)
```

Шаг 3. One-hot кодирование меток

```
from tensorflow.keras.utils import to_categorical

one_hot_train_labels = to_categorical(train_labels)
one_hot_test_labels = to_categorical(test_labels)
```

Шаг 4. Формирование валидационной выборки

Забираем часть обучающих данных:

```
x_val = x_train[:1000]
partial_x_train = x_train[1000:]

y_val = one_hot_train_labels[:1000]
partial_y_train = one_hot_train_labels[1000:]
```

Шаг 5. Построение модели softmax-классификации

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(10000,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax")
])

model.compile(optimizer="rmsprop",
              loss="categorical_crossentropy",
              metrics=["accuracy"])
```

Шаг 6. Обучение модели

```
history = model.fit(
```

```
partial_x_train, partial_y_train,  
epochs=20, batch_size=512,  
validation_data=(x_val, y_val)  
)
```

Шаг 7. Анализ графиков

Студент должен построить:

- график потерь (loss — на обучении и валидации),
- график точности (accuracy — на обучении и валидации).

И ответить:

1. С какого момента начинается переобучение?
2. На какой эпохе достигается максимум точности на валидации?
3. Какое число эпох является оптимальным?

Шаг 8. Обучение финальной модели

Используем выбранное число эпох (обычно 8–9):

```
final_model = keras.Sequential([...])  
final_model.fit(x_train, one_hot_train_labels, epochs=optimal_epochs,  
batch_size=512)
```

Шаг 9. Оценка на тестовой выборке

```
test_loss, test_acc = final_model.evaluate(x_test, one_hot_test_labels)  
print(test_acc)
```

Шаг 10. Прогнозирование

Модель выдаёт вектор вероятностей:

```
predictions = final_model.predict(x_test[:3])  
print(np.argmax(predictions[0]))
```

4. Практическая часть В — Регрессия (Boston Housing)

Шаг 1. Загрузка данных

```
from tensorflow.keras.datasets import boston_housing
```

```
(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()
```

Шаг 2. Нормализация признаков

```
mean = train_data.mean(axis=0)
std = train_data.std(axis=0)
```

```
train_data = (train_data - mean) / std
test_data = (test_data - mean) / std
```

Шаг 3. Построение модели регрессии

```
def build_model():
    model = keras.Sequential([
        layers.Dense(64, activation='relu', input_shape=(train_data.shape[1],)),
        layers.Dense(64, activation='relu'),
        layers.Dense(1)
    ])
    model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
    return model
```

Шаг 4. k-fold кросс-валидация

- выбрать $k = 4$,
- тренировать модель 100 эпох на каждом fold,
- сохранить значения MAE на валидации.

Построить график средней MAE по эпохам.

Шаг 5. Выбор оптимального числа эпох

Студент должен определить:

- при каком числе эпох MAE минимальна,
- когда начинается ухудшение,
- сколько эпох выбрать.

Шаг 6. Обучение финальной модели

```
model = build_model()
model.fit(train_data, train_targets, epochs=optimal_epochs, batch_size=16)
```

Шаг 7. Оценка на тестовой выборке

```
model.evaluate(test_data, test_targets)
```

Шаг 8. Прогнозирование

```
predictions = model.predict(test_data[:5])
```

Студент должен сравнить прогнозы с реальными ценами.

5. Исследовательские задания (выбрать любые 2, по одному из каждой части)

Часть А — Reuters

1. Изменить размер скрытых слоёв (32/64/128).
2. Добавить третий скрытый слой.
3. Использовать Dropout(0.5).
4. Изменить num_words (5000 / 20000).
5. Сравнить оптимизаторы (SGD / Adam).
6. Изменить размер валидационной выборки.
7. Использовать sparse_categorical_crossentropy вместо categorical_crossentropy.

Часть В — Boston Housing

8. Изменить архитектуру (количество слоёв).
9. Добавить L2-регуляризацию.
10. Добавить Dropout.
11. Изменить количество эпох и повторить k-fold.
12. Сравнить различные размеры batch_size.

6. Контрольные вопросы

Студент отвечает на любые 4 вопроса (по 2 из каждой части):

По части А (классификация):

1. Чем многоклассовая классификация отличается от бинарной?
2. Что делает softmax?
3. Почему используется categorical_crossentropy?
4. Как интерпретировать вектор вероятностей на выходе модели?
5. Как определить переобучение по графику loss?

По части В (регрессия):

6. Почему для регрессии выходной слой один и без активации?
7. Зачем нормализовать признаки?
8. Что показывает MAE?
9. В чём суть k-fold валидации?
10. Как по k-fold определить оптимальное число эпох?

7. Что сдаёт студент

В Jupyter Notebook:

- все шаги работы для Reuters и Boston Housing;
- все графики;
- выполненные 2 исследовательских задания;
- ответы на контрольные вопросы.

Краткий отчёт (1–2 страницы):

- цель работы,
- описание моделей,
- результаты (ассурасу для Reuters и MAE для Boston Housing),
- выводы по исследованиям,
- ответы на вопросы.

```
{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "# Практическое занятие 4.2\n## Многоклассовая классификация (Reuters) и регрессия (Boston Housing) с использованием нейронных сетей Keras\n\nВ этом практическом занятии мы рассмотрим два важных типа задач для нейронных сетей:\n1. **Многоклассовая классификация** на примере новостных сообщений (набор данных Reuters).\n2. **Регрессия** на примере предсказания стоимости жилья (набор данных Boston Housing).\n\nМы научимся:\n- подготавливать данные для многоклассовой классификации;\n- использовать one-hot кодирование меток;\n- строить сеть с выходом `softmax` и функцией потерь `categorical_crossentropy`;\n- решать задачу регрессии нейронной сетью;\n- применять нормализацию признаков;\n- выполнять k-fold кросс-валидацию для подбора числа эпох."
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "## 1. Подготовка окружения\n\nУбедимся, что установлены необходимые библиотеки:\n- `tensorflow` / `keras` – для построения и обучения моделей;\n- `numpy` – для работы с массивами;\n- `matplotlib` – для визуализации результатов.\n\nЕсли каких-то библиотек не хватает, их необходимо установить через `pip install ...`."
      ]
    },
    {
      "cell_type": "code",
      "metadata": {},
      "source": [
        "# 1. Импорт необходимых библиотек\n\nimport numpy as np\nimport matplotlib.pyplot as plt\n\nfrom tensorflow.keras.datasets import reuters, boston_housing\nfrom tensorflow import keras\nfrom tensorflow.keras import layers\nfrom tensorflow.keras.utils import to_categorical\n\nprint("Готово: библиотеки успешно импортированы.")"
      ],
      "outputs": [],
      "execution_count": null
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "# Часть А. Многоклассовая классификация новостей (набор данных Reuters)\n\nВ этой части мы решаем задачу многоклассовой классификации новостных сообщений:\n- вход: текст новости, представлен в виде
      ]
    }
  ]
}
```


последовательности индексов слов;\n- выход: номер темы (класса) новости.\n\nНабор данных **Reuters** содержит тысячи кратких новостных заметок, размеченных по темам (около 46 классов).\n"

```
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## 2. Загрузка и первичный анализ набора данных Reuters\n\nМы ограничим словарь `num_words = 10000` наиболее частыми словами. Данные автоматически разделены на обучающую и тестовую выборки.\n\nКаждый пример – это список целых чисел (индексов слов), а метка – целое число от 0 до числа классов - 1.\n"
```

```
]
},
{
  "cell_type": "code",
  "metadata": {},
  "source": [
    "# 2. Загрузка набора данных Reuters\n\nnum_words = 10000\n(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=num_words)\n\nprint("Количество обучающих примеров:", len(train_data))\nprint("Количество тестовых примеров:", len(test_data))\nprint("Пример закодированного текста (первые 20 индексов):", train_data[0][:20])\nprint("Метка первого примера:", train_labels[0])"
```

```
],
"outputs": [],
"execution_count": null
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## 3. Векторизация входных данных\n\nКак и в задаче IMDb, нам нужно превратить последовательности индексов в векторы фиксированной длины.\n\nМы будем использовать функцию `vectorize_sequences`, которая создаёт матрицу размера `(количество_примеров, num_words)`, где каждый пример представлен в виде многомерного one-hot вектора (bag-of-words).\n"
```

```
]
},
{
  "cell_type": "code",
  "metadata": {},
  "source": [
    "# 3. Векторизация входных данных\n\nfrom keras.preprocessing.sequence import vectorize_sequences\n\nsequences = list(range(1000))\nresults = np.zeros((len(sequences), dimension))\n\nvectorize_sequences(sequences, dimension=10000):\n\n    """Преобразует список последовательностей индексов слов в матрицу (len(sequences), dimension) с 0/1 значениями.\n\n    results = np.zeros((len(sequences), dimension))\n"
```

```

dimension), dtype="float32")\n    for i, seq in enumerate(sequences):\n
    results[i, seq] = 1.0\n    return results\n\nx_train =
vectorize_sequences(train_data, dimension=num_words)\nx_test =
vectorize_sequences(test_data, dimension=num_words)\n\nprint("\nФорма
x_train:", x_train.shape)\nprint("\nФорма x_test:", x_test.shape)"
],
"outputs": [],
"execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 4. One-hot кодирование меток (классов)\n\nМетки новостей – целые
числа: 0, 1, 2, ..., 45 (например).\n\nДля обучения с функцией потерь
`categorical_crossentropy` удобно перевести их в one-hot
представление:\nвектор длины `num_classes`, где на позиции соответствующего
класса стоит 1, а остальные 0.\n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 4. One-hot кодирование меток\n\none_hot_train_labels =
to_categorical(train_labels)\none_hot_test_labels =
to_categorical(test_labels)\n\nprint("\nПример one-hot метки для первого
примера:")\nprint(one_hot_train_labels[0])\nprint("\nФорма матриц меток:",
one_hot_train_labels.shape, one_hot_test_labels.shape)"
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 5. Формирование валидационной выборки\n\nВыделим часть обучающих
данных для валидации, чтобы отслеживать переобучение.\n\nПусть первые 1000
примеров будут использоваться для валидации, а остальные – для обучения
модели.\n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 5. Формирование валидационной выборки\n\nx_val =
x_train[:1000]\npartial_x_train = x_train[1000:]\n\ny_val =
one_hot_train_labels[:1000]\npartial_y_train =

```

```

one_hot_train_labels[1000:]\n\nprint("\nОбучающая выборка:",
partial_x_train.shape, y_val.shape)\nprint("\nВалидационная выборка:",
x_val.shape, y_val.shape)"
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 6. Построение модели многоклассовой классификации\n\nПостроим
модель с несколькими скрытыми слоями и выходным слоем `softmax`: \n\n- вход:
вектор длины 10 000;\n- скрытые слои: `Dense(64, activation='relu')`; \n-
выходной слой: `Dense(num_classes, activation='softmax')`; \n\nФункция потерь:
`categorical_crossentropy`, \nметрика: `accuracy`. \n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 6. Построение и компиляция модели для многоклассовой
классификации\n\nnum_classes = one_hot_train_labels.shape[1]\n\nmodel_reuters
= keras.Sequential([\n    layers.Dense(64, activation="\nrelu\n",
input_shape=(num_words,)), \n    layers.Dense(64, activation="\nrelu\n"), \n
    layers.Dense(num_classes,
activation="\nsoftmax\n")\n])\n\nmodel_reuters.compile(\n
optimizer="\nrmsprop\n", \n    loss="\ncategorical_crossentropy\n", \n
metrics=[\naccuracy\n"]\n)\n\nmodel_reuters.summary()"
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 7. Обучение модели и история обучения\n\nОбучим модель в течение
нескольких эпох и посмотрим, как меняются функция потерь и точность \nна
обучении и на валидации. \n\nПо графикам будет видно, начинается ли
переобучение. \n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 7. Обучение модели многоклассовой классификации\n\nhistory_reuters
= model_reuters.fit(\n    partial_x_train, \n    partial_y_train, \n

```

```

epochs=20,\n    batch_size=512,\n    validation_data=(x_val, y_val),\n    verbose=1\n)"
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 8. Визуализация результатов обучения (Reuters)\n\nПостроим
графики:\n\n1. Функция потерь на обучающей и валидационной выборках.\n2.
Точность на обучающей и валидационной выборках.\n\nНа основе графиков
ответьте на вопросы и сделайте выводы о переобучении и выборе числа эпох.\n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 8. Графики потерь и точности для задачи Reuters\n\nhistory_dict =
history_reuters.history\n\nloss = history_dict["loss"]\nval_loss =
history_dict["val_loss"]\nacc = history_dict["accuracy"]\nval_acc =
history_dict["val_accuracy"]\n\nepochs = range(1, len(loss) +
1)\n\nplt.figure(figsize=(12, 4))\n\n# График функции потерь\nplt.subplot(1,
2, 1)\nplt.plot(epochs, loss, "o-", label="Потери на
обучении")\nplt.plot(epochs, val_loss, "o-", label="Потери на
валидации")\nplt.title("Функция потерь
(Reuters)")\nplt.xlabel("Эпоха")\nplt.ylabel("Loss")\nplt.legend()\n\n#
График точности\nplt.subplot(1, 2, 2)\nplt.plot(epochs, acc, "o-",
label="Точность на обучении")\nplt.plot(epochs, val_acc, "o-",
label="Точность на валидации")\nplt.title("Точность классификации
(Reuters)")\nplt.xlabel("Эпоха")\nplt.ylabel("Accuracy")\nplt.legend()\n
\nplt.tight_layout()\nplt.show()"
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 9. Анализ переобучения (Reuters)\n\n**Задание для
студента:**\n\n1. По графику потерь определите, начиная с какой эпохи
валидационная ошибка перестаёт уменьшаться и начинает расти.\n2. По графику
точности найдите эпоху, на которой валидационная точность достигает
максимума.\n3. Сделайте вывод: сколько эпох обучения оптимально использовать
для задачи Reuters?\n\nНапишите ответы в виде небольшого текста (3–5
предложений) в отдельной ячейке Markdown.\n"
    ]
}
]

```

```

    },
    {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
            "## 10. Обучение финальной модели (Reuters) с оптимальным числом эпох\n\nВыберите оптимальное число эпох (например, 8 или 9) и обучите **новую модель** с нуля\nна всех обучающих данных (`x_train`, `one_hot_train_labels`).\nЗатем оцените качество на тестовой выборке (`x_test`, `one_hot_test_labels`).\n"
        ]
    },
    {
        "cell_type": "code",
        "metadata": {},
        "source": [
            "# 10. Обучение финальной модели Reuters с уменьшенным числом эпох\n\noptimal_epochs_reuters = 8 # при необходимости измените это значение по результатам анализа\n\nfinal_model_reuters = keras.Sequential([\n    layers.Dense(64, activation="relu", input_shape=(num_words,)),\n    layers.Dense(64, activation="relu"),\n    layers.Dense(num_classes, activation="softmax")\n])\n\nfinal_model_reuters.compile(\n    optimizer="rmsprop",\n    loss="categorical_crossentropy",\n    metrics=["accuracy"]\n)\n\nfinal_model_reuters.fit(\n    x_train,\n    one_hot_train_labels,\n    epochs=optimal_epochs_reuters,\n    batch_size=512,\n    verbose=1\n)\n\ntest_loss_reuters, test_acc_reuters = final_model_reuters.evaluate(x_test, one_hot_test_labels, verbose=0)\nprint(f"Точность финальной модели на тестовой выборке (Reuters): {test_acc_reuters:.4f}")
        ],
        "outputs": [],
        "execution_count": null
    },
    {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
            "## 11. Примеры прогнозов (Reuters)\n\nПосмотрим на прогнозы модели для нескольких примеров из тестовой выборки:\n\n- Модель выдаёт вектор вероятностей длины `num_classes`.\n- Класс с наибольшей вероятностью можно получить с помощью `argmax`.\n"
        ]
    },
    {
        "cell_type": "code",
        "metadata": {},
        "source": [
            "# 11. Примеры прогнозов для нескольких тестовых новостей\n\npredictions = final_model_reuters.predict(x_test[:5])\n\nfor i, p in enumerate(predictions):\n    predicted_class = np.argmax(p)\n"
        ]
    }

```

```

true_class = test_labels[i]\n    print(f"Пример {i}: предсказанный класс =
{predicted_class}, истинный класс = {true_class}\")\n"
],
"outputs": [],
"execution_count": null
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"# Часть В. Регрессия: предсказание стоимости жилья (Boston
Housing)\n\nТеперь рассмотрим задачу **регрессии**, в которой модель должна
предсказывать\nне класс, а числовое значение – в нашем случае среднюю
стоимость жилья в районе.\n\nМы будем использовать набор данных **Boston
Housing**, который содержит:\n\n- признаки районов (налоги, качество школ,
криминогенная обстановка и т.п.); \n- целевая переменная – средняя стоимость
домов в тысячах долларов.\n"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"## 12. Загрузка и анализ набора данных Boston Housing\n\nНабор
данных автоматически разделён на обучающую и тестовую выборки.\n\nКаждый
пример представлен в виде вектора признаков (около 13 признаков), а целевая
переменная – одно число (цена).\n"
]
},
{
"cell_type": "code",
"metadata": {},
"source": [
"# 12. Загрузка набора данных Boston Housing\n\n(train_data_boston,
train_targets_boston), (test_data_boston, test_targets_boston) =
boston_housing.load_data()\n\nprint("\nФорма train_data_boston:\n",
train_data_boston.shape)\nprint("\nФорма test_data_boston:\n",
test_data_boston.shape)\nprint("\nПример вектора признаков:\n",
train_data_boston[0])\nprint("\nПример целевой переменной:\n",
train_targets_boston[0])"
],
"outputs": [],
"execution_count": null
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"## 13. Нормализация признаков\n\nНейронные сети, как правило, лучше
обучаются, если входные признаки нормализованы.\n\nМы приведём каждый признак

```

к виду: (значение - среднее) / стандартное_отклонение, \n используя статистику по обучающей выборке. \n"

```
    ],
    },
    {
        "cell_type": "code",
        "metadata": {},
        "source": [
            "# 13. Нормализация признаков\n\nmean =
train_data_boston.mean(axis=0)\nstd =
train_data_boston.std(axis=0)\n\ntrain_data_boston_norm = (train_data_boston
- mean) / std\n\ntest_data_boston_norm = (test_data_boston - mean) /
std\n\nprint("\nСреднее по обучающей выборке (после нормализации должно быть
около
0):\n")\nprint(train_data_boston_norm.mean(axis=0))\nprint("\n\nСтандартное
отклонение (после нормализации должно быть около
1):\n")\nprint(train_data_boston_norm.std(axis=0))"
        ],
        "outputs": [],
        "execution_count": null
    },
    {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
            "## 14. Построение модели для регрессии\n\nДля регрессии мы построим
небольшую полносвязную сеть:\n\n- Несколько скрытых слоёв с `relu`;\n-
Выходной слой `Dense(1)` без функции активации (линейный выход);\n- Функция
потерь: `mse` (mean squared error);\n- Основная метрика: `mae` (mean absolute
error) – средняя абсолютная ошибка.\n"
        ],
        },
    {
        "cell_type": "code",
        "metadata": {},
        "source": [
            "# 14. Функция для построения модели регрессии\n\ndef
build_regression_model():\n    model = keras.Sequential([\n
        layers.Dense(64, activation=\n\n\"relu\",
input_shape=(train_data_boston_norm.shape[1],)),\n
        layers.Dense(64,
activation=\n\n\"relu\"),\n
        layers.Dense(1) # линейный выход\n    ])\n
    model.compile(\n
        optimizer=\n\n\"rmsprop\",
        loss=\n\n\"mse\",
        metrics=[\n\n\"mae\"])\n
    return model\n\nmodel_reg =
build_regression_model()\nmodel_reg.summary()"
        ],
        "outputs": [],
        "execution_count": null
    },
    {
        "cell_type": "markdown",

```

```

    "metadata": {},
    "source": [
        "## 15. k-fold кросс-валидация для подбора числа эпох\n\nНабор Boston Housing небольшой, поэтому мы будем использовать **k-fold кросс-валидацию**:\n\n1. Разделим обучающие данные на `k` частей (например, `k = 4`).\n2. Для каждого `fold`:\n    - используем 1 часть как валидацию;\n    - оставшиеся `k-1` частей – для обучения;\n    - обучаем модель несколько эпох и фиксируем метрику на валидации.\n3. Усредним результаты по всем `k` fold.\n\nТак можно оценить, какое число эпох даёт наилучший результат.\n"
    ]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 15. k-fold кросс-валидация\n\nk = 4\nnum_val_samples = len(train_data_boston_norm) // k\nnum_epochs = 100\nall_mae_histories = []\n\nfor i in range(k):\n    print(f"Обработка fold #{i}")\n    # Формируем валидационный и обучающий наборы\n    val_data = train_data_boston_norm[i * num_val_samples : (i + 1) * num_val_samples]\n    val_targets = train_targets_boston[i * num_val_samples : (i + 1) * num_val_samples]\n    \n    partial_train_data = np.concatenate(\n        [train_data_boston_norm[: i * num_val_samples],\n        train_data_boston_norm[(i + 1) * num_val_samples :]],\n        axis=0\n    )\n    partial_train_targets = np.concatenate(\n        [train_targets_boston[: i * num_val_samples],\n        train_targets_boston[(i + 1) * num_val_samples :]],\n        axis=0\n    )\n    \n    # Строим и обучаем модель\n    model = build_regression_model()\n    history = model.fit(\n        partial_train_data,\n        partial_train_targets,\n        epochs=num_epochs,\n        batch_size=16,\n        validation_data=(val_data, val_targets),\n        verbose=0\n    )\n    \n    mae_history = history.history["val_mae"]\n    all_mae_histories.append(mae_history)\n\nprint("Готово: кросс-валидация завершена.")
    ]
},
{
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 16. Анализ результатов k-fold кросс-валидации\n\nУсредним значения `val_mae` (средняя абсолютная ошибка на валидации) по всем fold\nдля каждой эпохи и построим график.\n\nЗатем по графику оценим, после какого числа эпох качество перестает улучшаться.\n"
    ]
},
{
    "cell_type": "code",

```



```
"metadata": {},  
    "source": [  
        "# 16. Усреднение метрик и построение графика  
MAE\n\naverage_mae_history = [\n    np.mean([x[i] for x in  
all_mae_histories]) for i in  
range(num_epochs)\n]\n\nplt.figure(figsize=(6,4))\nplt.plot(range(1,  
num_epochs + 1), average_mae_history, \"o-  
\")\nplt.xlabel(\"Эпоха\")\nplt.ylabel(\"Средняя MAE на  
валидации\")\nplt.title(\"k-fold: средняя MAE на валидации (Boston  
Housing)\")\nplt.show()\n    ],  
    "outputs": [],  
    "execution_count": null  
},  
{  
    "cell_type": "markdown",  
    "metadata": {},  
    "source": [  
        "## 17. Выбор оптимального числа эпох и обучение финальной  
модели\n\n**Задание для студента:**\n\n1. Посмотрите на график средней MAE по  
эпохам.\n2. Определите диапазон эпох, в котором MAE минимальна (качество  
наилучшее).\n3. Выберите число эпох (например, 80) и обучите финальную модель  
на *всех* обучающих данных.\n\nПосле этого оцените качество на тестовой  
выборке (`test_data_boston_norm`, `test_targets_boston`)."\n    ],  
},  
{  
    "cell_type": "code",  
    "metadata": {},  
    "source": [  
        "# 17. Обучение финальной модели регрессии с выбранным числом  
эпох\n\noptimal_epochs_boston = 80 # при необходимости измените это значение  
по результатам анализа\n\nfinal_model_reg =  
build_regression_model()\nfinal_model_reg.fit(\n    train_data_boston_norm,\n    train_targets_boston,\n    epochs=optimal_epochs_boston,\n    batch_size=16,\n    verbose=0\n)\n\ntrain_mse, test_mae =  
final_model_reg.evaluate(test_data_boston_norm, test_targets_boston,  
verbose=0)\nprint(f"Средняя абсолютная ошибка (MAE) на тестовой выборке  
(Boston Housing): {test_mae:.2f} тыс. долларов")\n    ],  
    "outputs": [],  
    "execution_count": null  
},  
{  
    "cell_type": "markdown",  
    "metadata": {},  
    "source": [  
        "## 18. Примеры прогнозов (Boston Housing)\n\nПосмотрим, какие  
значения стоимости жилья предсказывает модель для нескольких
```

примеров.\n\nНапомним: целевая переменная выражается в **тысячах долларов**.\n"

```
]
},
{
    "cell_type": "code",
    "metadata": {},
    "source": [
        "# 18. Примеры прогнозов стоимости жилья\n\npredictions_boston =\nfinal_model_reg.predict(test_data_boston_norm[:5])\n\nfor i, pred in\n    enumerate(predictions_boston):\n        true_value = test_targets_boston[i]\n        print(f"Пример {i}: предсказанная цена = {pred[0]:.2f} тыс. $, истинная\n        цена = {true_value:.2f} тыс. $")
    ],
    "outputs": [],
    "execution_count": null
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 19. Исследовательские задания (индивидуальная работа)\n\nКаждый студент выполняет **минимум два** исследовательских задания (по выдаче преподавателя).\n\n### Варианты для части А (Reuters):\n\n1. Изменить размер скрытых слоёв (например, 32 и 32 вместо 64 и 64) и сравнить качество.\n2. Добавить третий скрытый слой и оценить влияние на переобучение.\n3. Использовать `Dropout` в скрытых слоях (например, 0.5) и сравнить результаты.\n4. Изменить размер словаря (`num_words = 5000` или `num_words = 20000`) и оценить влияние.\n5. Попробовать другой оптимизатор (`adam`, `sgd`) и сравнить динамику обучения.\n6. Изменить размер валидационной выборки (например, 500 или 2000 примеров).\n7. Попробовать `sparse_categorical_crossentropy` без one-hot кодирования меток (оставив их целыми числами).\n\n### Варианты для части В (Boston Housing):\n\n8. Изменить архитектуру модели (количество и размер скрытых слоёв) и оценить влияние на MAE.\n9. Добавить L2-регуляризацию к весам (`kernel_regularizer=keras.regularizers.l2(0.001)`) и сравнить результаты.\n10. Попробовать `Dropout` в одном из слоёв и оценить его влияние.\n11. Изменить количество эпох и заново провести k-fold кросс-валидацию (например, 50 или 150 эпох).\n12. Изменить размер батча (8, 32) и оценить влияние на MAE и устойчивость обучения.\n\n**Требования к отчёту по исследовательским заданиям:**\n\n- описать, какие изменения были внесены;\n- привести графики (loss/accuracy или MAE);\n- сделать содержательные выводы (5–10 предложений) по результатам экспериментов.\n"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [

```

```
    "## 20. Контрольные вопросы\n\nОтветьте письменно (в отдельной ячейке Markdown или в итоговом отчёте) на **не менее четырёх** вопросов:\n\n### По части A (Reuters):\n\n1. Чем многоклассовая классификация отличается от бинарной с точки зрения архитектуры и функции потерь?\n2. Что делает функция активации `softmax` на выходном слое?\n3. Почему для многоклассовой классификации используется `categorical_crossentropy`?\n4. Как интерпретировать выход модели – вектор вероятностей длиной `num_classes`?\n5. Как можно выявить переобучение по графикам loss/accuracy?\n\n### По части B (Boston Housing):\n\n6. Чем задача регрессии отличается от задачи классификации с точки зрения выходного слоя и функции потерь?\n7. Зачем нормализовать входные признаки перед обучением модели?\n8. Что показывает метрика MAE? В каких единицах она измеряется в этой задаче?\n9. В чём идея k-fold кросс-валидации и зачем её применять?\n10. Каким образом можно определить оптимальное число эпох обучения по результатам k-fold?\n\nОтветы должны быть развёрнутыми и понятными для одноклассника, который не присутствовал на занятии.\n"
```

```
    ]\n    },\n    {\n        "cell_type": "markdown",\n        "metadata": {},\n        "source": [\n            "## 21. Итог работы и оформление отчёта\n\nПо результатам выполнения практического занятия 4.2 студент должен подготовить:\n\n1. **Заполненный Jupyter Notebook**, содержащий:\n    - все этапы подготовки данных и построения моделей;\n    - результаты обучения (графики и числовые метрики);\n    - реализацию исследовательских заданий (не менее двух);\n    - ответы на контрольные вопросы.\n\n2. **Краткий текстовый отчёт** (можно в виде итоговой ячейки Markdown), включающий:\n    - цель и задачи работы;\n    - описание использованных моделей;\n    - основные числовые результаты (точность на Reuters и MAE на Boston Housing);\n    - выводы по проведённым экспериментам;\n    - ответы на контрольные вопросы.\n\nНа следующем занятии можно будет использовать результаты этой работы для более глубокого понимания настройки и анализа моделей глубокого обучения.\n"
```

```
    ]\n    },\n    ],\n    "metadata": {\n        "language_info": {\n            "name": "python",\n            "pygments_lexer": "ipython3"\n        },\n        "orig_nbformat": 4\n    },\n    "nbformat": 4,\n    "nbformat_minor": 5\n}
```